# Python Dictionaries — Solutions & Explanations

Class XI Computer Science

Computer Science Department

October 21, 2026

# Question 1: Sorting a Dictionary

**Task:** Sort the dictionary `{'ravi':'10', 'rajnish':'9', 'sanjeev':'15', 'yash':'2', 'suraj':'32'}` and print it.

```
d = {'ravi': '10', 'rajnish': '9', 'sanjeev': '15', 'yash': '2', 'suraj': '32'}
# 1. Sorted by keys (alphabetical order)
sorted_by_keys = dict(sorted(d.items()))
print("Sorted by Keys:", sorted_by_keys)
# 2. Sorted by values (numerical order, casting str to int)
sorted_by_values = dict(sorted(d.items(), key=lambda item: int(item[1])))
print("Sorted by Values:", sorted_by_values)
```

## Key Takeaway

Dictionaries in modern Python ($\geq 3.7$) preserve insertion order. Passing `sorted(d.items())` to `dict()` yields an ordered dictionary.

# Question 2: Highest 2 Values in a Dictionary

**Task:** Find the two highest values present in a dictionary.

```
marks = {'Physics': 88, 'Maths': 99, 'CS': 97, 'Chemistry': 92, 'English': 85}
# Extract unique values, sort in descending order, slice top 2
unique_values = sorted(set(marks.values()), reverse=True)
top_2 = unique_values[:2]
print("Original Dictionary:", marks)
print("Highest 2 Values:", top_2)
```

## Note

Using `set()` ensures duplicate highest values (e.g., two subjects having 99) do not occupy both the first and second slots if unique highest tiers are desired.

# Question 3: Character Frequency Dictionary

**Task:** Create a frequency count dictionary from the string `'w3resource'`.

```
text = 'w3resource'
freq = {}
for char in text:
    freq[char] = freq.get(char, 0) + 1
print("Character Frequencies:")
print(freq)
```

## Explanation

The `dict.get(key, default)` idiom retrieves the existing tally if present; otherwise, it returns 0, avoiding `KeyError`.

# Question 4: Employee Salaries Input

**Task:** Input $n$ employee names and salaries, storing them in a dictionary.

```
n = int(input("Enter number of employees: "))
employees = {}
for i in range(n):
    name = input(f"Enter name of employee {i + 1}: ")
    salary = float(input(f"Enter salary for {name}: "))
    employees[name] = salary
print("\n--- Employee Records ---")
for name, salary in employees.items():
    print(f"{name}: Rs. {salary:.2f}")
```

# Question 5: Count Character Appearances

**Task:** Count how many times each character appears in a given string.

```
s = input("Enter a string: ")
counts = {}
for ch in s:
    if ch in counts:
        counts[ch] += 1
    else:
        counts[ch] = 1
for char, count in counts.items():
    print(f"'{char}' : {count}")
```

## Alternative

Can also be implemented concisely using Python's standard `collections.Counter(s)`.

# Question 6: Number to Words Converter

**Task:** Convert a user-entered number (e.g., 876) to words (`'Eight Seven Six'`).

```
digit_map = {
    '0': 'Zero',  '1': 'One', '2': 'Two',   '3': 'Three', '4': 'Four',
    '5': 'Five',  '6': 'Six', '7': 'Seven', '8': 'Eight', '9': 'Nine'
}
num_str = input("Enter a number: ")
# Map each digit character using dictionary lookup
words = [digit_map[d] for d in num_str if d in digit_map]
result = " ".join(words)
print("Output:", result)
```

# Question 7: Sports Team Win/Loss Records

**Task:** Store team stats as `{team: [wins, losses]}` and answer queries.

```
teams = {}
n = int(input("How many teams? "))
for _ in range(n):
    name = input("Team name: ")
    w = int(input("Wins: "))
    l = int(input("Losses: "))
    teams[name] = [w, l]
# (a) Winning percentage
query = input("\nEnter team to query win percentage: ")
if query in teams:
    w, l = teams[query]
    total = w + l
    pct = (w / total * 100) if total > 0 else 0.0
    print(f"{query} Winning Percentage: {pct:.2f}%")
# (b) List of win counts
wins_list = [record[0] for record in teams.values()]
print("Wins across all teams:", wins_list)
# (c) Teams with winning records (wins > losses)
winning_teams = [t for t, (w, l) in teams.items() if w > l]
print("Teams with winning records:", winning_teams)
```

# Question 8: Product Price Lookup

**Task:** Store products/prices, then allow repeated search lookups.

```
catalog = {}
print("Enter product data (type 'done' to stop):")
while True:
    prod = input("Product name: ").strip()
    if prod.lower() == 'done':
        break
    price = float(input(f"Price for {prod}: "))
    catalog[prod] = price
print("\n--- Product Search (type 'exit' to quit) ---")
while True:
    query = input("Search product: ").strip()
    if query.lower() == 'exit':
        break
    if query in catalog:
        print(f"Price of {query}: Rs. {catalog[query]:.2f}")
    else:
        print(f"'{query}' is not available in the catalog.")
```

# Question 9: Month Days & Analysis

**Task:** Dictionary of month names and day counts with analytical lookups.

```
months = {
    'January': 31, 'February': 28, 'March': 31,    'April': 30,
    'May': 31,     'June': 30,     'July': 31,     'August': 31,
    'September': 30,'October': 31, 'November': 30, 'December': 31
}
# (a) Query days
m = input("Enter month name: ").capitalize()
print(f"Days in {m}:", months.get(m, "Month not found"))
# (b) Keys in alphabetical order
print("Alphabetical:", sorted(months.keys()))
# (c) Months with exactly 31 days
print("31-day months:", [m for m, days in months.items() if days == 31])
# (d) Key-value pairs sorted by days
print("Sorted by days:", sorted(months.items(), key=lambda x: x[1]))
```

# Question 10: Multi-Field Records in a Dictionary

**Answer: Yes.** A dictionary can store composite records by using roll numbers as unique keys and nested dictionaries or lists/tuples as the values.

```
# Storing 10 students using roll number as primary key
students = {
    101: {'name': 'Aarav', 'marks': 94.5, 'grade': 'A1'},
    102: {'name': 'Bhavna', 'marks': 88.0, 'grade': 'A2'},
    103: {'name': 'Chirag', 'marks': 76.5, 'grade': 'B1'},
    # ... up to 10 students
    110: {'name': 'Jaya', 'marks': 91.0, 'grade': 'A1'}
}
# Accessing attributes directly
print("Roll 101 Name:", students[101]['name'])
print("Roll 101 Grade:", students[101]['grade'])
```

# Question 11: Inverting a Dictionary

**Task:** Map `x = {'k1':'v1', 'k2':'v2', 'k3':'v3'}` to reversed key-value pairs.

```
x = {'k1': 'v1', 'k2': 'v2', 'k3': 'v3'}
# Using dictionary comprehension
inverted_x = {value: key for key, value in x.items()}
print("Original:", x)
print("Inverted:", inverted_x)
```

## Constraint

This approach assumes that all values in the original dictionary are unique and hashable (e.g., strings, ints, tuples).

# Question 12: Overlapping Keys Between Dictionaries

**Task:** List keys common to both `D1` and `D2`.

```
D1 = {'a': 1, 'b': 2, 'c': 3, 'd': 4}
D2 = {'b': 20, 'd': 40, 'e': 50}
# Method 1: Set intersection (idiomatic and O(min(len(D1), len(D2))))
common_keys = list(D1.keys() & D2.keys())
print("Overlapping keys (Set Intersection):", common_keys)
# Method 2: Standard loop
overlapping = [k for k in D1 if k in D2]
print("Overlapping keys (List Comprehension):", overlapping)
```

# Question 13: Detecting Duplicate Values

**Task:** Check if distinct keys hold duplicate values.

```
def check_duplicate_values(d):
    vals = list(d.values())
    if len(vals) != len(set(vals)):
        print(f"For {d}: 2 keys have same values")
    else:
        print(f"For {d}: No keys have same values")
D1 = {'a': 10, 'b': 20, 'c': 10}
D2 = {'a': 10, 'b': 20, 'c': 30}
check_duplicate_values(D1)
check_duplicate_values(D2)
```

## Logic

If `len(values) > len(set(values))`, at least one value appears more than once across different keys.

# Question 14: Sub-Dictionary Verification

**Task:** Check if dictionary d1 is completely contained within d2.

```
d1 = {1: 11, 2: 12}
d2 = {1: 11, 2: 12, 3: 13, 4: 15}
# Method 1: Using set operations on .items()
is_contained = d1.items() <= d2.items()
print("d1 is contained in d2:", is_contained)
# Method 2: Explicit key-value verification
contained = all(k in d2 and d2[k] == v for k, v in d1.items())
print("Verified via generator:", contained)
```

# Question 15: Summing List Elements as Values

**Task:** Compute sum of lists in `D1 = {'A': [1, 2, 3], 'B': [4, 5, 6]}` into `D2`.

```
D1 = {'A': [1, 2, 3], 'B': [4, 5, 6]}
# Dictionary comprehension with built-in sum()
D2 = {key: sum(values) for key, values in D1.items()}
print("Original D1:", D1)
print("Computed D2:", D2)
```

# Question 16: Phonebook Operations Manager

**Task:** Interactive friend phone directory handling all standard operations.

```
phonebook = {'Ananya': '9847012345', 'Rohan': '9447123456'}
# (a) Display all
print("(a) All contacts:", phonebook)
# (b) Add new contact
phonebook['Devika'] = '9895011223'
print("(b) Added Devika:", phonebook)
# (c) Delete a friend
phonebook.pop('Rohan', None)
print("(c) Deleted Rohan:", phonebook)
# (d) Modify phone number
if 'Ananya' in phonebook:
    phonebook['Ananya'] = '9744000000'
print("(d) Modified Ananya:", phonebook)
# (e) Check membership
print("(e) Is Ananya present?", 'Ananya' in phonebook)
# (f) Sorted by name
print("(f) Sorted contacts:", dict(sorted(phonebook.items())))
```